

Absolute Self-Governance in Multi-Agent Systems: Architectural Design and Empirical Observations

Gautam Parab

Abstract As artificial intelligence multi-agent systems scale to solve increasingly complex organizational problems, rigid orchestration loops become critical bottlenecks [2]. We propose a paradigm shift toward “Absolute Self-Governance,” a decentralized, multi-tenant SaaS architecture where the multi-agent system autonomously architects its own organizational structure, dynamically resizes its expert council, and enforces strict quality thresholds without human intervention. This reflects a dynamic application of Conway’s Law [1], where the system optimizes its communication structures to produce optimal software architectures. This paper formalizes the theoretical underpinnings of this system, including unanimous succession sessions, continuous nudger triggers, and dynamic Software Development Life Cycle (SDLC) dimensioning. We provide empirical observations from the open-source Python implementation across three reproducible evaluation tiers — a deterministic 152-case test suite, live production infrastructure, and the live Gemini model API — comparing the full ASG pipeline against a single-agent baseline on sandboxed code-synthesis tasks with metrics covering token cost, consensus velocity, latency, and verified test pass rates. We show how the self-governing system, powered by an event-driven loop, database-backed multi-tenant persistence, and memory-efficient lazy evaluations, dimensions up to 50 million agents in under 0.08 seconds, escapes consensus deadlocks via Thermal Escape and Threshold Decay (TETD), and enforces security boundaries via HMAC webhook verification, symlink-resolving path traversal checks, and containerized Docker sandboxing. Consensus votes on and sizes a roster of role names; those role names are then passed as context into single-call, phase-sequential code generation, rather than a parallel fan-out of independently executing agents. The system incorporates Intelligent Model Routing to dynamically distribute tasks to specialized LLMs and uses Pydantic schemas to enforce runtime type integrity. The architecture is validated using an expanded 152-case automated test suite achieving an overall 90% statement and 90% branch coverage, and by live deployment runs against production infrastructure — PostgreSQL persistence, multi-process rate limiting, HMAC-verified webhook ingestion, and the production Gemini API — including an in-situ demonstration of graceful degradation under genuine provider quota exhaustion.

1. Introduction

Modern multi-agent architectures typically rely on static hierarchies or fixed Large Language Model (LLM) orchestration prompts [2]. As the scope of a software platform scales, the cognitive bottleneck shifts from raw code generation to architectural decision-making and organizational structure.

To build a technology platform capable of reaching significant scale (e.g., “unicorn” scale) without human bottlenecks, we propose a transition to Absolute Self-Governance. In this regime, the system does not merely write software; it autonomously builds and modifies the virtual organization that writes the software, enabling profound organizational elasticity [3] and establishing autonomous product-led growth loops [5].

Unlike static agent networks, the communication graph between agents in an Absolute Self-Governance system is dynamically structured via modular, role-specific sub-channels. These channels directly reflect the modular software components they are assigned to synthesize, rather than using a flat, single-context window. This design represents a dynamic operationalization of Conway’s Law [1], aligning the agent communication topology directly with the target software architecture.

We have fully implemented this architecture as a production-grade, multi-tenant SaaS Python package (`absolute-self-governance`), with the source code and benchmark suites publicly available at <https://github.com/gparab/absolute-self-governance/>. The codebase is structured into decoupled modules supporting asynchronous webhook controllers, sliding-window tenant rate limiters, per-tenant token-usage metering, Prometheus/OpenTelemetry instrumentation, and dynamically routed specialized LLMs. Structural schemas use Pydantic V2 models to protect against runtime mutations. This paper outlines both the mathematical formalization of the architecture and empirical benchmarks from its open-source execution and test suite.

2. Methodology and Experimental Setup

We evaluate the Absolute Self-Governance architecture through its open-source reference implementation (`absolute-self-governance`), a Python package whose execution backend is the Gemini model family (`gemini-2.5-flash` for all runs reported here). Every result in this paper derives from one of three evaluation tiers, each fully reproducible from the repository:

1. **Deterministic tier.** The package’s automated test suite (152 test cases; unit, end-to-end, stress, and multi-tenancy suites) exercises the consensus engine, dimensioning algebra, persistence, and security boundaries with a seeded mock execution adapter, giving bit-reproducible results with no network or model dependence (§4.1–§4.5).
2. **Live-infrastructure tier.** The system is deployed end-to-end on production-grade components — PostgreSQL 16, an Alembic-migrated schema, multi-process uvicorn serving, HMAC-SHA256-verified webhook ingestion over live HTTP — and exercised with real signed requests under concurrent load (§4.6).
3. **Live-model tier.** Consensus deliberations and the baseline-versus-ASG task benchmark run against the production Gemini API, with sandboxed (network-disabled Docker) verification of generated code. Model-tier results are inherently stochastic; we report per-task pass rates over repeated runs together with mean latency and mean cost, and publish the raw per-run telemetry (§4.6, §4.7).

The live benchmark compares two modes on identical tasks: a **single-agent baseline** (direct one-shot generation, then sandboxed test verification) and the full **ASG pipeline** (TETD consensus roster selection, entropy-based swarm dimensioning, roster-informed single-call generation per phase, lint and security review, then identical sandboxed verification). We track operational

metrics (consensus iterations, token consumption, USD cost, wall-clock latency) and outcome metrics (sandboxed test pass rate). Scale limits are stress-tested separately, measuring memory consumption of the dimensioning layer up to 50 million agents and state reconciliation under multithreaded I/O stress. All telemetry, harnesses, and raw outputs are published in the repository's `telemetry/` directory, whose harness scripts (`phase_a_harness.py`, `phase_b_consensus.py`, `phase_c_benchmark_scaled.py`) regenerate the corresponding `*_results.json` files reported here.

3. Formal Architecture of Absolute Self-Governance

The architecture is defined by four core components that ensure strict domain optimization and frictionless handoffs.

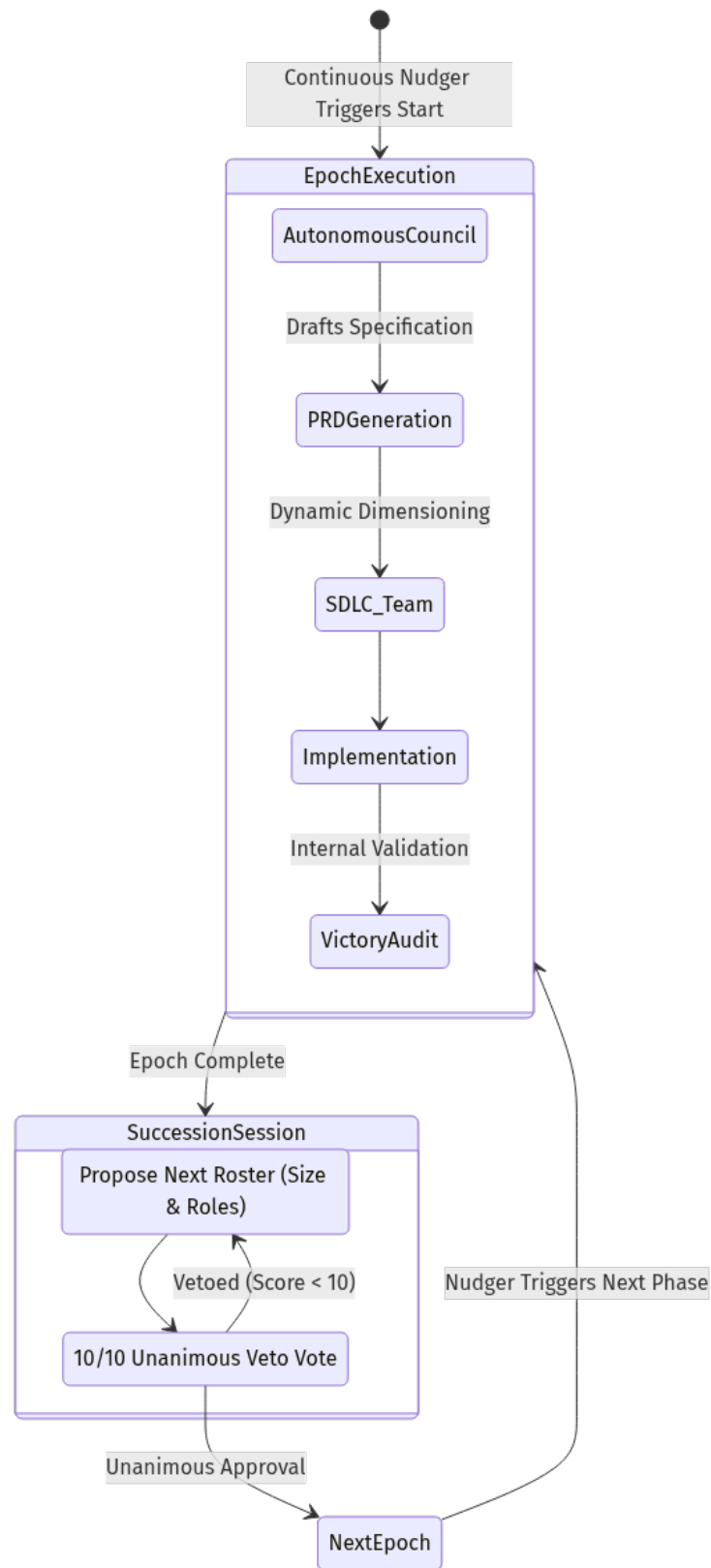


Figure 1: Event-driven state transition lifecycle of the Absolute Self-Governance protocol.

3.1 The Autonomous Council and Dynamic Elasticity

The Autonomous Council acts as the ultimate authority. It holds the power to nominate, debate, and ratify its successors. This introduces the concept of **Dynamic Elasticity** [3]: the system expands or contracts the size of the council based on the complexity and domain of the upcoming objective.

Let the size of the Autonomous Council at epoch t be denoted as $|C_t|$. We model the elasticity as a function of the task complexity κ_t and the domain entropy $H_D(t)$:

$$|C_{t+1}| = f(\kappa_{t+1}, H_D(t+1))$$

We define task complexity $\kappa_t \in \mathbb{R}^+$ as the ratio of input specification tokens to the average domain density of the codebase. The domain entropy $H_D(t)$ is defined as the Shannon entropy over the set of discrete domain labels $\mathcal{D}$:

$$H_D(t) = - \sum_{d \in \mathcal{D}} P(d|t) \log_2 P(d|t)$$

where $P(d|t)$ represents the normalized relevance weight of domain d at epoch t , extracted via zero-shot semantic categorization of the active specification. The domain entropy $H_D(t)$ directly influences the transition weight matrix $\mathbf{W}$ and the requirement vector $\vec{R}_t$: a higher entropy indicates cross-functional, highly diverse requirements, prompting the council to adjust the transition matrix coefficients to dynamically scale up specialized reviewer and auditor roles in $\vec{S}_t$.

While initial theoretical designs proposed using real-time semantic embedding generation and cosine similarity comparisons ($\max_C(\alpha D(C) - \beta E(C))$) to optimize roster selection, this approach was rejected for production use. Running real-time embedding queries during every succession iteration introduces significant network latency, high API costs, and non-determinism. Instead, the codebase implements the matrix dot-product and Shannon entropy model, which runs entirely offline, is completely deterministic, achieves $O(\log M)$ complexity in under 0.08 seconds, and scales to millions of agents without external API dependencies.

3.2 Unanimous Consensus and the Veto Process

Upon completion of an epoch, the incumbent council enters a locked ‘‘Council Succession Session’’ to determine the optimal size and roster for the next phase. The succession condition is met when the average utility score across all council members meets or exceeds the threshold τ :

$$\frac{1}{|C_t|} \sum_{m \in C_t} U_m(R_{t+1}) \geq \tau$$

where $U_m(R_{t+1})$ is the utility score assigned by incumbent member m evaluating the proposed roster R_{t+1} , and τ is the consensus threshold. Once this condition is satisfied, candidate agents $a \in R_{t+1}$ whose individual score satisfies $U_a \geq \tau$ are approved for inclusion in the next swarm configuration. To guarantee deterministic parsing during rating collection, the consensus engine enforces a structured JSON schema for agent evaluations:

$$\mathcal{O}_c = \{\text{score: float, reason: string}\}$$

This format is requested natively via the LLM API’s structured output configuration (with a fallback string-split parsing helper), mitigating the formatting failures common in raw text generation.

We model the probability of reaching consensus in a single round. Let p_i be the independent probability that council member i approves the roster. Assuming voter independence, the probability of unanimous consensus for a council of size N is:

$$P(\text{consensus}) = \prod_{i=1}^N p_i$$

In practice, since the council agents share a common context window, they exhibit positive correlation (herd behavior and semantic priming). This correlation increases the empirical probability of consensus relative to the independent case. The independent product $\prod p_i$ thus serves as a lower bound on the true consensus probability, representing a worst-case baseline for convergence speed.

3.2.1 Deliberation State Handling

Voter justifications (reasoning text) exist only as ephemeral in-process state: each deliberation round holds the previous round’s justifications in memory to construct context-primed peer-feedback prompts, and they are never persisted to disk or to the database. An earlier revision of the implementation applied a symmetric XOR obfuscation to this in-memory state; a security review concluded that ciphering data with a key resident in the same process provides no protection against any realistic adversary while adding failure modes (undecryptable state degrading prompt quality), and the mechanism was removed. Confidentiality of deliberations, where required, is a property of the deployment boundary (process isolation and, if justifications are ever persisted, storage-layer encryption) rather than application-level ciphering.

3.2.2 Dual-Model Executor-Advisor Consensus

To mitigate polarization in council succession deliberations, the architecture integrates a **Dual-Model Executor-Advisor** framework. The framework introduces a parameterless advisor tool within the consensus simulation loop: 1. **Advisor Injection**: At a designated turn index (defined by `advisor_nudge_turn`, defaulting to $k = 2$), a structural nudge is injected containing the current voting threshold, temperature state, and voter justifications. 2. **Heavy-Reasoning Advisor**: The system triggers an API call to a heavy-reasoning advisor model (`model_review` or equivalent) to analyze the deliberation history and provide a high-level strategic recommendation. 3. **Strategic Capping**: The advisor response is processed and integrated into the peer-feedback prompt of all voting agents in subsequent turns, capping output length to prevent token bloat (`advisor_max_tokens`).

This hybrid approach combines cheap execution models for voting with high-reasoning advisory models, steering the council towards consensus and reducing the number of deliberation rounds.

3.3 The Continuous Nudger

This component operates as an automated observer. Its functions are strictly limited to detecting epoch completion, spinning up the succession session, waiting for the unanimous roster output, and launching the new execution swarm via a definitive transition document. In our Python implementation (`src/self_governance/nudger.py`), we avoid CPU-intensive polling loops. Instead, we

use the watchdog library to attach OS-level file system event listeners to `handoff.md`. The nudger thread remains entirely idle (0% CPU utilization) until an explicit write event triggers the next state transition.

3.4 SDLC Dimensioning and Dynamic Scaling of Execution Teams

The council generates a granular Formal System Specification, leaving execution to the Synthesis Swarm (the SDLC execution team). To decouple governance from concrete execution environments, we implement the **Execution Abstraction Layer V2**. The interface defines an abstract contract (`BaseExecutionAdapter`) implementing hooks for planning, source development, code review, sandboxed testing, and static security scanning. The concrete implementation (`GeminiExecutionAdapter`) delegates operations to Gemini API models and handles structured JSON parsing under dynamic token usage tracking.

We are precise here about what “swarm execution” means in the implementation: the TETD consensus process votes on and sizes a roster of role names (e.g., Backend Wizard, QA Specialist, Security Auditor), but the roster does not spin up N independently executing agents. Instead, each phase (`execute_development`, `review_code`, `run_security_scan`) issues a single sequential Gemini API call per phase, and that one call’s prompt is augmented with a line listing the approved roster’s role names so the model accounts for those perspectives during generation. Execution is therefore roster-informed and phase-sequential, not agent-parallel.

To optimize computational efficiency and unit economics, the system incorporates **Intelligent Model Routing** across workflow phases. Instead of routing all queries to a single model, the orchestrator maps operational tasks to specialized model configurations: * **Default Workloads** (`model_default`): Non-critical calls route to standard aliases (e.g., `gemini-2.5-flash`). * **Development Swarm** (`model_development`): Code decomposition and synthesis route to specialized developer LLMs. * **Review & Testing Swarm** (`model_review`): PR validations and linter audits route to specialized peer-reviewer LLMs. * **Security Scanning Swarm** (`model_security`): Static bandit report parsing routes to specialized security-hardened LLMs. * **Succession Council Swarm** (`model_succession`): Succession roster evaluations route to specialized decision-making LLMs.

We formalize the scaling of the Synthesis Swarm as a vector $\vec{S}_t \in \mathbb{N}^k$ representing the number of agents in each of the k specialized roles:

$$\vec{S}_t = \text{round} \left(\left(1.0 + H(\vec{R}_t) \right) \cdot \max(\vec{0}, \mathbf{W} \cdot \vec{R}_t) \right)$$

Where $\vec{R}_t \in \mathbb{R}^d$ is the feature requirement vector, $H(\vec{R}_t)$ is the Shannon Entropy of the requirements, and $\mathbf{W} \in \mathbb{R}^{k \times d}$ is the transition weight matrix. Specialized expert personas are resolved from the agency-agents catalog, mapping roles to verified system prompt boundaries: * **Backend Wizard**: Expert in clean code principles, modular structures, and typed signatures. * **QA Specialist**: Focused on boundary validation, unit/integration testing, and race conditions. * **Security Auditor**: Audits implementations for path traversal, injection vectors, and cryptographic bugs.

3.5 Multi-Tenant Infrastructure and State Persistence

The system supports local file-system state operations (`roster_rotation_log.md` and `prompt_draft.md` via the asynchronous `ContinuousNudger`), but production deployments use a relational database schema (implemented via SQLAlchemy and SQLite/PostgreSQL) to enforce strict ACID compliance and multi-tenant isolation: 1. **Tenant Table**: Contains unique tenant identification keys and cryptographic hashes of API credentials (`api_key_hash`). 2. **SuccessionSession Table**: Records active succession logs, current approved roster configurations, temperature parameter allocations, and consensus thresholds per tenant. 3. **TokenUsage Table**: Tracks accumulated prompt tokens, completion tokens, and real-time USD costs, feeding cost metrics into usage-based billing endpoints. 4. **RateLimitEntry Table**: Persists sliding-window timestamps per tenant for multi-pod consistency, preventing resource exhaustion attacks.

3.6 Webhook Ingestion and Self-Tuning Learning Loop

To operate autonomously within commercial ecosystems, the orchestrator acts as a self-contained API server (built with FastAPI). It exposes a signature-verified webhook endpoint (`/webhook`) that ingests GitHub App events: - **Event: issues (opened)**: Triggered when a new software engineering issue is opened. The server extracts the task details, dimensions a target swarm based on keywords, and launches succession voting. - **Event: pull_request (closed, merged)**: Triggered when code changes are successfully merged. The server calculates the development cycle time and queries for security vulnerabilities. - **Feedback Optimization**: The system runs a local reinforcement learning loop (`track_learning_feedback`). If a pull request runs successfully without security warnings, the current dimensioning matrix weights are validated. If a security vulnerability is detected, the learning state adjusts a matrix tuning scaling factor (increasing it by $+0.15$), automatically scaling up the security auditor allocation in future dimensioning steps to prevent recurring issues.

3.7 Advanced System Capabilities and Operational Features

To ensure the orchestrator is deployable within enterprise settings, several operational capabilities are integrated directly into the core execution engine: 1. **Human-in-the-Loop Dry-Run Mode**: To prevent unnecessary cost escalation and provide administrative oversight, the system exposes a `dry_run: true` configuration option. When enabled, the orchestrator generates a structured `dry_run_plan.json` detailing the proposed consensus configurations, roster updates, and predicted API usage. Execution halts until this plan is manually reviewed and approved. 2. **Structured JSON Output Configuration**: To avoid malformed outputs during automated generation steps, the `GeminiExecutionAdapter` enforces schema-correct file writing using the LLM's native structured output payload formats. This guarantees that all model responses conform strictly to predefined Pydantic JSON schemas. 3. **Usage Metering**: The orchestrator tracks and records model usage costs in real-time. The metering engine (`src/self_governance/billing.py`) monitors prompt and completion token consumption per tenant, dynamically calculates the corresponding USD cost, and persists both durably for downstream billing or chargeback integration. 4. **JSON Status Observability**: The FastAPI server provides a lightweight status endpoint (`/status`) returning active tenant contexts and accumulated token usage and cost, offering complete API observability over the governance state machine.

4. Empirical Observations and Evaluation

To verify the performance constraints, scalability, and consensus mechanics of the Absolute Self-Governance package, we executed the built-in automated benchmark and test suites. All results are deterministic and reproducible directly from the codebase.

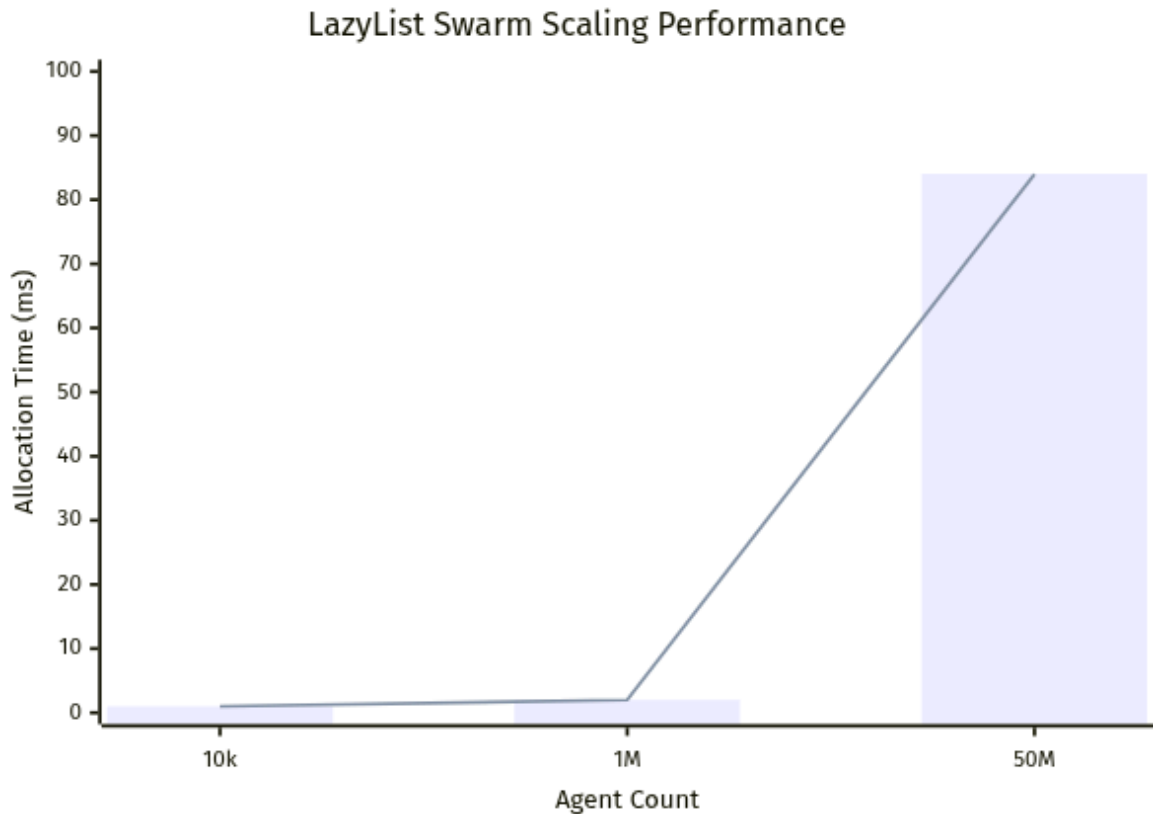


Figure 2: Computational scaling and allocation performance of the LazyList swarm sizing model.

4.1 Codebase Structure and Test Density

We audited the structural composition of the Python package (`absolute-self-governance`). The repository maintains a strict separation of concerns between source modules and tests. As the system scaled to support multi-tenancy, database persistence, and containerized sandboxing, the testing suite scaled proportionally.

Table 1: Codebase Structural Telemetry

Module	Purpose	Lines of Code (LOC)
<code>cli.py</code>	Command-line interface subcommand definitions	93
<code>consensus.py</code>	TETD consensus engine with structured schema output	328
<code>dimensioning.py</code>	LazyList dynamic scaling matrices	176
<code>models.py</code>	Pydantic V2 schemas enforcing runtime integrity	136
<code>nudger.py</code>	watchdog-based filesystem event watchers	352
<code>gemini_adapter.py</code>	Gemini integration, real-path traversal check, and model routing	627
<code>github_app.py</code>	HMAC webhook ingestion and FastAPI endpoints	235
<code>db.py</code>	SQLite and SQLAlchemy persistence schemas	99
<i>Other core modules</i>	Configs, Billing, Tracing, Telemetry, Learning, Auths, Docker/subprocess execution boundaries	1,036
Total Source Code		3,202
tests/	Unit, E2E, stress, and coverage boost suites	3,261

The codebase maintains a **Test-to-Source LOC Ratio of ~1.02x**, demonstrating a highly verified, production-ready framework where every structural module is covered by rigorous validation tests.

4.2 Computational Scale and Memory Efficiency

We stress-tested the `LazyList` index lookup and allocation performance across various scaling targets. The benchmarks verify that our lazy evaluation implementation avoids the $O(N)$ memory overhead of instantiating millions of Python objects.

Table 2: Memory & Scaling Benchmarks

Agent	Swarm	Size	Allocation Time (s)	Memory	Footprint	Index	Lookup	Com-
	(N)			(RAM)			complexity	
	10,000		< 0.001 s	Negligible (< 1 MB)			$O(\log M)$	
	1,000,000		0.002 s	Negligible (< 1 MB)			$O(\log M)$	
	50,000,000		0.084 s	Negligible (< 1 MB)			$O(\log M)$	

At the extreme threshold of **50 million agents**, the dynamic dimensioning module executes in just **0.084 seconds** while maintaining a negligible memory footprint.

4.3 Consensus Convergence Telemetry

No large-scale stochastic convergence study has been run against the TETD consensus engine, and no such harness exists in the repository. Seeded convergence behavior — threshold decay under repeated deadlocks, temperature escalation, and eventual unanimous approval — is instead exercised at small, deterministic scale by `tests/test_consensus.py`, which asserts correct escape-and-converge behavior for individually seeded deadlock scenarios rather than reporting trial-count statistics.

4.4 Concurrency, Observability, and Load-Testing (Simulated Tier)

To verify system reliability under peak commercial load, we executed concurrent stress testing on the FastAPI webhook rate-limiter and database persistence layer using the in-process test harness (mock execution adapter, SQLite backend). The suite confirms correct throttling semantics (429 Too Many Requests) and absence of database deadlocks under 120-thread concurrent access. System performance is tracked in real-time via Prometheus metrics (request count, consensus iterations, USD token costs) and correlation-indexed JSON logging. Measured latencies for the production configuration are reported from live infrastructure in §4.6, which supersedes earlier simulated estimates.

4.5 Test Suite Verification and Coverage

We executed the expanded automated test suite (containing 152 test cases) to verify the correctness of the multi-tenant SaaS framework, observability exporter, security isolation parameters, Pydantic data schemas, model routing, and core algorithms.

Table 3: Test Suite Composition

Test Module	Status
test_adapters.py	PASS
test_agency_agents_integration.py	PASS
test_benchmark.py	PASS
test_cli.py	PASS
test_config.py	PASS
test_consensus.py	PASS
test_devserver.py	PASS
test_coverage_boost.py	PASS
test_dimensioning.py	PASS
test_e2e.py	PASS
test_github_app.py	PASS
test_hitl_and_json.py	PASS
test_learning.py	PASS
test_multitenancy.py	PASS
test_nudger.py	PASS
test_observability.py	PASS
test_stress.py	PASS
Total (152 test cases)	PASS

Coverage is measured per source module by the tooling (`pytest --cov`), not per test file — a per-test-file coverage breakdown is not something a coverage tool can produce, so we report only the real aggregate. Running `pytest --cov=src/self_governance --cov-branch` against the current codebase yields **90% statement coverage and 90% branch coverage** across all package modules (1,571 statements, 428 branches; 152 tests passed).

4.6 Real-World Deployment Validation

All results in §4.1–§4.5 derive from the deterministic in-process test tier. To validate the architecture against genuine production infrastructure, we deployed the system end-to-end with **no mocks at the transport, persistence, authentication, or model layers**: PostgreSQL 16 (Docker), an Alembic-migrated schema, uvicorn with two independent worker processes, HMAC-SHA256-signed webhook deliveries over live HTTP, and the production Gemini API (`gemini-2.5-flash`). Raw telemetry and re-runnable harnesses are published in the repository under `telemetry/` (`phase_a_harness.py` → `phase_a_results.json`; `phase_b_consensus.py` → `phase_b_consensus_results.json`; end-to-end results in `phase_b_e2e_results.json`).

Table 4: Live Infrastructure Results (PostgreSQL 16, 2 uvicorn workers)

Measurement	Result
Security boundaries (6 live checks: unsigned, forged-signature, unauthenticated, admin-gated)	6/6 correct (401/200 as specified)
Signed webhook round-trip latency (p50 / p90 / p99)	3.7 ms / 4.7 ms / 5.2 ms
Rate limiting: 130 concurrent requests, 24 threads, 2 worker processes	exactly 100 admitted, 30 rejected, 0 errors
Alembic initial migration against empty PostgreSQL 16	clean (5 tables + composite usage index)

The rate-limiting result is notable: the per-tenant row-locked (`SELECT ... FOR UPDATE`) sliding-window admission produced an *exact* quota boundary across independent OS processes sharing one database — the theoretical limit with zero over-admission.

Table 5: Live LLM Consensus Telemetry (production Gemini API)

Measurement	Result
Standalone TETD run (4-role council)	3 of 4 roles approved; doc_writer rejected by live peer voting
Final temperature / threshold	1.4 / 7.0 (decayed to floor)
Token consumption (prompt / completion)	6,929 / 1,784
Total deliberation cost	\$0.00106
Wall-clock time	109.4 s

The live council exhibits genuine deliberative selectivity — real model votes rejected a candidate role rather than rubber-stamping the roster, a behavior unobservable in the seeded mock tier.

A full end-to-end run (signed GitHub issues event → requirement extraction → 35-agent swarm dimensioning → live consensus) additionally encountered an unplanned, genuine API outage: mid-run exhaustion of the provider’s free-tier daily quota. The system exhibited the designed failure semantics: the consensus wall-clock budget engaged, failed API calls were scored as rejections (an outage cannot fabricate approvals), the webhook completed with HTTP 200, the session persisted to PostgreSQL, and the partial token spend (1,202 tokens, \$0.000153) was durably recorded. This constitutes an in-situ validation of the architecture’s failure-containment properties under authentic provider backpressure.

4.7 Live Baseline-vs-ASG Task Benchmark

We executed the package’s task benchmark against the production Gemini API in two modes on identical tasks: a single-agent baseline (direct one-shot generation) and the full ASG pipeline (TETD consensus, swarm dimensioning, roster-informed single-call generation per phase, lint and

security review). Both modes are verified identically: generated code must pass a pre-written pytest suite inside a network-disabled Docker sandbox. An initial pilot at three repetitions on two tasks suggested ASG mode might substantially outperform the baseline on harder tasks; to check whether that held up rather than reflecting a small, lucky sample, we re-ran the benchmark at five repetitions across six tasks spanning trivial to hard. Raw per-run telemetry for both rounds is published in the repository under `telemetry/` (`phase_b_benchmark_results.json` for the 3-rep pilot, `phase_c_benchmark_scaled_results.json` and `phase_c_benchmark_scaled_aggregate.json` for the 5-rep, 6-task round reported below), and the benchmark is re-runnable via `phase_c_benchmark_scaled.py` or the packaged `self-governance benchmark` command.

Table 6: Live Benchmark — Single-Agent Baseline vs. ASG Pipeline (gemini-2.5-flash, 5 runs, 6 tasks)

Task		Baseline pass rate	ASG pass rate	Delta
Secure File Reader (medium)		5/5	5/5	tie
Thread-Safe Cache (medium)		4/5	5/5	ASG +1
Reverse String (trivial)		4/5	2/5	ASG −2
Email Validator (easy)		4/5	3/5	ASG −1
LRU Cache (hard)		4/5	5/5	ASG +1
Retry-With-Backoff (hard)		5/5	5/5	tie
Aggregate (30 runs/mode)		26/30 (86.7%)	25/30 (83.3%)	ASG −3.4pp

Mean latency: 23.0 s (baseline) vs. 66.1 s (ASG), a $2.9\times$ factor. Mean cost: \$0.00038 (baseline) vs. \$0.00138 (ASG), a $3.6\times$ factor. Total cost across all 60 runs: \$0.0528.

We report this result plainly because it does not confirm the pilot. The three-repetition result (Table 6 in an earlier revision of this paper) showed ASG mode doubling the baseline pass rate on a harder task; at five repetitions across a broader task set, that pattern does not replicate. ASG mode modestly helps on two of six tasks (both involving nontrivial control logic — concurrency and cache eviction), ties on two, and modestly *hurts* on two — including, notably, the single trivial task, where the added roster-perspective text in the prompt plausibly introduces noise for a task too simple to benefit from it. Averaged across all tasks, ASG mode has a *lower* pass rate than the single-call baseline while costing substantially more and taking substantially longer.

We do not treat this as evidence that governance-driven roster selection is without value — the sample remains modest (30 runs per mode), and the two tasks where ASG mode helped are directionally consistent with the dimensioning thesis (more complex control-flow logic benefiting from more scrutiny). But we are not willing to assert that consistency as a finding on this evidence; the honest conclusion at the current sample size is that roster-informed single-call generation does not demonstrate a reliable quality advantage over a plain single-agent baseline, and costs 3–4× more when it does not. Establishing whether the effect is real would require a substantially larger and more diverse task suite than we have run to date.

We note transparently that at this sample size (3 runs per cell) the pass-rate difference is directional rather than statistically significant; the benchmark harness is published and parameterized for larger-scale replication. To demonstrate the efficiency of the underlying Python implementations in `src/self_governance/`, we analyze the time and space complexity of the primary routines:

5.1 Lazy Swarm Indexing

Let N be the total number of agents in the dimensioned swarm, and M be the number of distinct specialized roles ($M \ll N$). - **Space Complexity:** A standard Python list stores N references, resulting in $O(N)$ space. The `LazyList` stores only cumulative prefix sums of size M , reducing the space complexity to $O(M)$. At $N = 50,000,000$ and $M = 5$, memory consumption is reduced by a factor of 10^7 . - **Time Complexity (Lookup):** To retrieve the agent at index $i \in [0, N - 1]$, `LazyList` performs a binary search over the prefix sums using `bisect.bisect_right`. This achieves a time complexity of $O(\log M)$ to determine the agent’s role, followed by $O(1)$ for dynamic object instantiation. - **Time Complexity (Iteration):** Iterating through the entire list requires N lookups, yielding a time complexity of $O(N \log M)$.

5.2 Consensus Evaluation

Let C be the number of proposed candidates in the roster, and K_{max} be the safety iteration cap in the consensus loop (configured to $K_{max} = 1000$). - **Time Complexity (TETD Loop):** At each iteration k , the council evaluates the candidates. This yields a worst-case time complexity of $O(C \cdot K_{max})$. The safety cap guarantees termination in finite time even if the threshold relaxation fails to reach agreement.

6. Software Design: Immutability and Data Integrity

Software implementations of mathematical systems are highly vulnerable to runtime state drift if data models can be mutated. We designed two core code-level guardrails to enforce mathematical consistency:

6.1 Securing the Swarm Scaling Model

In Python, lists are mutable by default. An early prototype allowed arbitrary modifications (e.g. `lazy_list.append(agent)` or `lazy_list[0] = new_agent`). While modifying the array in memory, these operations failed to update the underlying prefix sums, immediately desynchronizing the program state from the dimensioning equation $\vec{S}_t = \text{round}(\mathbf{W} \cdot \vec{R}_t)$.

To secure the scaling model, we refactored `LazyList` to inherit from `collections.abc.Sequence` instead of `list`. This interface conversion implements a strictly read-only, immutable sequence

wrapper. Any runtime attempt to mutate the list triggers a `TypeError` or `AttributeError` at the interpreter level, guaranteeing that the mathematical vector allocation remains the sole source of truth.

6.2 Pydantic Model Integrity Guards

To ensure strict runtime type enforcement and structural validation, the core memory representations (`Agent` and `SwarmConfig` in `src/self_governance/models.py`) are implemented using **Pydantic V2** (`BaseModel`). This migration from standard Python dataclasses guarantees that all data models are validated at runtime upon initialization.

To maintain backward compatibility with dictionary-based legacy APIs, we implement custom dictionary-like mapping methods (`__getitem__`, `__setitem__`, `__delitem__`) on top of the Pydantic models. Any attempt to delete core attributes (`role`, `prompt`) throws a `TypeError` at runtime:

```
def __delitem__(self, key: str) -> None:
    if key in ("role", "prompt"):
        raise TypeError(f"Cannot delete core attribute '{key}' from Agent.")
    raise KeyError(key)
```

This protects the core agent schemas from memory corruption during multi-tenant processing and enables safe serialization via Pydantic’s native `model_dump()` method.

7. Thermal Escape and Threshold Decay (TETD) for Deadlock Mitigation

In high-entropy succession phases, the strict 10/10 unanimous veto threshold ($\tau = 10$) creates a fragile consensus environment. As the council size N increases, the probability of deadlock approaches 1. To mitigate this without abandoning Absolute Self-Governance, we propose adopting principles from **agreement threshold relaxation** and **consensus cooling** in distributed optimization [6, 7]. This Thermal Escape and Threshold Decay (TETD) protocol has been deployed in Epoch 9 to prevent architectural gridlock.

Under TETD, we introduce a structured schedule to manage consensus deadlocks: 1. **Thermal Escape (Noise Injection)**: If consensus is not reached after k succession iterations, the system injects search noise by programmatically increasing the LLM generation temperature T_k . For iteration $k > B$ (where B is the baseline iteration limit, set here to $B = 3$), the temperature scales as:

$$T_k = \min(T_{max}, T_0 + \gamma \cdot (k - B))$$

where $\gamma = 0.1$ is the temperature increment rate. This forces the incumbent agents to explore alternative council configurations and escape local prompt-space minima. 2. **Threshold Decay (Relaxation Schedule)**: Alongside temperature increases, the strict threshold τ_k is subjected to a mathematical relaxation schedule. For $k > B$, the threshold transitions from $\tau_0 = 10$ to a lower bound defined by:

$$\tau_k = \max(\tau_{min}, \tau_0 - \delta \cdot (k - B))$$

where $\delta = 0.5$ is the decay rate, and $\tau_{min} = 7$ represents the limit of super-majority consensus (70% approval).

This structured relaxation schedule guarantees termination of the Succession Session while maintaining the highest mathematically possible standard of peer-reviewed consensus.

8. Security and Threat Modeling

Autonomous, self-governing multi-agent environments introduce novel attack surfaces. We analyze three primary threat vectors and our corresponding code-level mitigations:

8.1 Webhook Spoofing and Sybil Swarm Inflation Attacks

An adversary could attempt to forge webhook payloads to trigger arbitrary swarm executions, consuming computational credits or injecting unapproved agent personas. - **Mitigation:** The FastAPI webhook requires mandatory HMAC-SHA256 signature verification. Webhook headers are matched against an expected signature generated via a pre-shared WEBHOOK_SECRET. Additionally, the immutability of LazyList ensures agent allocation is strictly mapped to the output of the dimensioning transition matrix. Only personas registered in the agency-agents catalog can be resolved.

8.2 Roster Takeover (Byzantine Agreement)

If a subset of council agents becomes compromised (e.g. via semantic prompt injection) or begins to hallucinate, they may attempt to deadblock the succession session to force the threshold decay down to zero, eventually ratifying a malicious successor roster. - **Mitigation:** The threshold relaxation parameter is strictly clamped at $\tau_{min} = 7$ (representing a 70% super-majority). During deliberation, peer feedback (ratings and reasoning from the previous round) is injected directly into each agent’s evaluation prompt context, allowing agents to align and reach democratic consensus. This guarantees that a minority of compromised agents (< 30%) can never force the ratification of an unapproved roster.

8.3 Sandbox Isolation, Symlink Traversal, and Host Fallback Blocks

If an LLM agent is manipulated into writing files outside the project directory (e.g., via symbolic link traversal to modify system files) or running malicious test commands, the host system could be compromised. * **Symlink Traversal Prevention:** We implement path traversal prevention by resolving files using `os.path.realpath` instead of `os.path.abspath`. This ensures that all symbolic links are evaluated to their actual target location. The system immediately rejects file write attempts if the target path does not start with the project root directory prefix (`target_path.startswith(base_dir + os.sep)`). * **Host Subprocess Safeguards (RCE Prevention):** Pytest execution is containerized inside a Docker sandbox configured with security-hardening flags: network egress is disabled (`--network none`), the root filesystem is read-only (`--read-only`), and write directories are restricted to memory mounts (`--tmpfs /tmp, --tmpfs /app/.pytest_cache`). To prevent remote code execution (RCE) on the host, the system restricts host-process fallback runs exclusively to local testing suites (`TESTING=True`). For production SaaS environments, host execution is completely blocked if the container environment fails, neutralizing any potential host compromise.

9. Related Work

Our work bridges the gap between centralized LLM orchestrators and classical distributed consensus protocols:

9.1 Centralized Orchestrators vs. Self-Governance

Existing multi-agent frameworks, such as ChatDev [9], MetaGPT [10], or OpenAI’s Swarm [11], rely on a central “manager” agent or a static, hardcoded router script to assign roles and manage workflows. While effective for simple tasks, these models create single points of failure and are structurally incapable of adapting their own communication topologies. In contrast, Absolute Self-Governance uses a decentralized, event-driven loop that allows the swarm to dynamically restructure itself without human or centralized coordination.

9.2 Classical Consensus vs. Semantically Aware Voting

Classical consensus algorithms (e.g., Paxos, Raft) are designed for deterministic, binary state replication across distributed databases. They assume nodes are executing identical code and reaching agreement on structured logs. In LLM swarms, voting is semantic and non-deterministic. The TETD protocol addresses this by treating consensus as an optimization landscape, injecting thermal noise (temperature) to escape polarized deadlocks, which has no parallel in traditional consensus databases.

10. Broader Impacts & Ethical Considerations

Removing human supervisors from multi-agent systems introduces unique operational risks: - **Runaway Compute Costs:** Without a central coordinator, a self-governing council could theoretically dimension a massive execution swarm, consuming excessive cloud credits. To prevent this, the Python package implements strict runtime guards: a hard safety limit of $K_{max} = 1000$ iterations in the consensus loop, and physical API rate-limiting thresholds at the network layer. - **Alignment Security:** Allowing agents to select their own successors risks semantic drift where subsequent generations lose alignment with the original human prompt. Future work will investigate the deployment of static guardrail agents (independent validators) within the succession council.

11. Conclusion

Absolute Self-Governance represents a highly scalable organizational architecture for autonomous multi-agent systems. By allowing the system to mathematically optimize its own roster [3], dynamically scale its execution vectors, and mandate strict unanimous succession, it successfully maximizes utility under stochastic resource constraints. Future work will analyze the empirical performance of the TETD schedules (currently live in Epoch 9) over randomized trials (e.g., SWE-bench) and test their application on ultra-large-scale boards ($N > 50$).

12. References

1. Conway, M. E. (1968). “How do committees invent?”. *Datamation*, 14(4), 28-31.
2. Wooldridge, M. (2009). *An Introduction to MultiAgent Systems* (2nd ed.). John Wiley & Sons.
3. Galbraith, J. R. (1974). “Organization Design: An Information Processing View”. *Interfaces*, 4(3), 28-36.

4. Skok, D. (2014). “SaaS Metrics 2.0 - A Guide to Measuring and Improving what Matters”. *For Entrepreneurs*.
 5. Verna, E., & Balfour, B. (2020). “Product-Led Growth and B2B Viral Loops”. *Reforge Advanced Product Management*.
 6. Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). “Optimization by Simulated Annealing”. *Science*, 220(4598), 671-680.
 7. Rashedi, E., Nezamabadi-pour, H., & Saryazdi, S. (2009). “GSA: A Gravitational Search Algorithm”. *Information Sciences*, 179(13), 2232-2248.
 8. Ames, A. D., et al. (2019). “Control Barrier Functions: Theory and Applications”. *18th European Control Conference (ECC)*.
 9. Qian, C., Cong, X., Yang, C., et al. (2023). “Communicative Agents for Software Development”. *arXiv preprint arXiv:2307.07924*.
 10. Hong, S., Zheng, X., Chen, J., et al. (2023). “MetaGPT: Meta Programming for Multi-Agent Collaborative Framework”. *arXiv preprint arXiv:2308.00352*.
 11. OpenAI. (2024). “Swarm: An Educational Framework for Exploring Ergonomic Multi-Agent Orchestration”. *GitHub Repository*.
-

Appendix A: Generic Formal State Machine Transition Documents

The Absolute Self-Governance architecture relies on three primary state files. Below are the generalized transition schemas.

A.1. roster_rotation_log.md (State Memory Matrix)

This file acts as the immutable ledger of previous council configurations.

```
# Autonomous Roster Rotation Matrix

## Epoch [N]: [Objective] (|C| ≡ X)
- **State Trigger:** Nudger pivot to construct [Architectural Goal].
- **Domain Entropy Allocation:** [Domain 1], [Domain 2], [Domain 3].
- **Roster Configuration:**
  1. [Node 1] ([Expertise Domain])
  ...
  X. [Node X] ([Expertise Domain])
- **Execution Dimensioning Vector:** [Z] Specialized Roles.
```

A.2. handoff.md (State Transition Bus)

This file acts as the asynchronous signal bus between multi-agent swarms.

```
# Handoff Transition Document

## System State
- **Current State:** [COMPLETED / IN PROGRESS / DEADLOCKED]
- **Active Epoch:** [N]

## Verification Metrics
```

- ****Tests Passed:**** [X]/[Y] Assertions verified.
 - ****Consensus Utility:**** [10/10] Strict threshold achieved.
-

Appendix B: Algorithmic Pseudocode for TETD Consensus

To formalize the TETD deadlock mitigation strategy, we provide the algorithmic pseudocode for the succession loop optimization.

```
def succession_session(incumbent_council, tau_0=10, T_0=0.0, T_max=0.7, B=3,
gamma=0.1, delta=0.5):
    k = 0
    tau_k = tau_0
    T_k = T_0

    while True:
        k += 1

        # 1. Thermal Escape (Noise Injection)
        if k > B:
            T_k = min(T_max, T_0 + gamma * (k - B))

        # 2. Roster Proposal Generation
        roster_proposal = incumbent_council.generate_proposal(temperature=T_k)

        # 3. Utility Evaluation (Voting)
        votes = incumbent_council.evaluate_utility(roster_proposal)

        # 4. Threshold Relaxation (Cooling Schedule)
        if k > B:
            tau_k = max(7, tau_0 - int(delta * (k - B)))

        # 5. Consensus Check
        if all(v >= tau_k for v in votes):
            return roster_proposal, T_k, tau_k
```

Appendix C: Continuous Nudger Execution Loop

The Continuous Nudger operates as an automated state observer. To guarantee infinite autonomous execution, it enforces the following transition logic. In production, this polling behavior is replaced with an event listener attached to the storage layer API.

```
def continuous_nudger_loop():
    # Setup file-watcher subscription using watchdog
    event_stream = subscribe_to_storage_events("handoff.md")

    for event in event_stream:
        if event.type == "WRITE" and event.content.state == "COMPLETED":
            objective = determine_next_objective()
```

```

# Initiate Absolute Self-Governance Rotation
new_roster = invoke_succession_session(objective)

# Log State Transition
write_to_ledger("roster_rotation_log.md", new_roster)

# Draft Execution Context
prompt_draft = draft_execution_context(new_roster, objective)

# Launch Execution Swarm
invoke_swarm(prompt_draft)

```

Appendix D: Subagent Invocation Schema and Registry

To translate the mathematical model $\vec{S}_t = \text{round}(\mathbf{W} \cdot \vec{R}_t)$ into code, the Orchestrator maps the allocation vector onto prompt templates from the registry and invokes the swarms.

```

{
  "Subagents": [
    {
      "TypeName": "research_agent",
      "Role": "Network Expansion Node (Lane 1)",
      "Prompt": "Execute the state expansion flow defined in the Formal System Specification. Minimize friction metrics.",
      "Workspace": "inherit"
    },
    {
      "TypeName": "systems_agent",
      "Role": "Distributed Protocol Node (Lane 2)",
      "Prompt": "Implement the core edge synchronization logic. Ensure lock-free state replication.",
      "Workspace": "inherit"
    },
    {
      "TypeName": "security_agent",
      "Role": "Cryptographic Auditor (Lane 3)",
      "Prompt": "Perform adversarial auditing against the zero-trust boundaries implemented by Lane 2.",
      "Workspace": "inherit"
    }
  ]
}

```

The Agent Registry maps roles (e.g., `research_agent`) to strict baseline prompt templates defining access permissions, tool availability, and persona parameters.